

Day 6

8/6 목

render 를 안 불러도 화면이 바뀌는 날

01

JSX 와 컴포넌트로 화면을 나눌 수 있다

02

useState 로 상태를 두고 render 없이 화면을 갱신할 수 있다

03

useEffect 로 불러오고 저장하고 뒷정리할 수 있다

오늘의 흐름

오전에 React, 오후에 인터넷에 올리기

오전, 이론 09:00 - 12:50

■ 09:00	왜 React 인가
■ 10:00	JSX 와 컴포넌트
■ 11:00	목록 그리기와 props
■ 12:00	useState 와 컴포넌트의 일생

오후, 배포 14:00 - 17:50

■ 14:00	GitHub 가입과 저장소
■ 15:00	파일 올리고 Pages 켜기
■ 16:00	확인하고 다듬기
■ 17:00	서면 회고, 클로징

01 왜 React 인가

어제 만드신 걸 다시 봅시다

새 탭 할 일 목록

- 01 상태가 하나 있었습니다. todos 배열이요
- 02 그리고 render 함수가 하나 있었습니다
- 03 추가할 때 save 하고 render 했습니다
- 04 체크할 때도, 지울 때도 save 하고 render 했습니다

오늘 이것 React 로 다시 만듭니다

화면은 똑같습니다. 안이 달라집니다

render 를 몇 번 부르셨나요

어제 코드에서 세어봅시다

- 01 addTodo 안에서 한 번
- 02 toggleTodo 안에서 한 번
- 03 removeTodo 안에서 한 번
- 04 toggleFilter 안에서 한 번. 그리고 맨 아래에서 한 번

다섯 군데입니다

기능을 하나 더 붙이면 여섯 군데가 됩니다

하나라도 빠뜨리면

어제 직접 겪으셨습니다

- 01 콘솔에서 todos 에 push 만 했을 때 기억나시죠
- 02 화면은 그대로였습니다. 상태만 바뀌었으니까요
- 03 기능이 늘어날수록 빠뜨릴 확률이 올라갑니다
- 04 그리고 빠뜨려도 에러가 안 납니다. 그냥 화면이 안 바뀝니다

상태를 바꿈

todos 에 하나 넣음



render 를 안 부름

화면은 그대로



에러도 안 남

왜 안 되는지 한참 찾음

어제 코드에서 render 를 지워보기

```
function addTodo(text) {  
  todos.push({ text: text, done: false });  
  save();  
  // render();    ← 이 줄만 주석 처리  
}
```

할 일

어제 만든 newtab.js 에서 한 줄만 막고 확장을 새로고침

생각해볼 것

추가는 됐을까요. 저장은 됐을까요

남은 시간

05:00

그리고 다시 그리는 방식의 한계

항목이 많아지면

01 어제 render 는 innerHTML 로 목록을 통째로 지우고 다시 만들었습니다

02 항목이 500개면 500개를 매번 다시 만듭니다

03 하나만 체크해도 전부 다시 만듭니다

04 입력하던 칸에 커서가 있었다면 그것도 날아갑니다

하나 바뀌었는데 전부 다시 만든다

이게 어제 방식의 진짜 문제입니다

React 가 대신 해주는 것

딱 두 가지입니다

- 01 상태가 바뀌면 다시 그리는 것을 알아서 합니다. render 를 부를 필요가 없습니다
- 02 그리고 전부 다시 만들지 않고 바뀐 부분만 고칩니다
- 03 그래서 커서도 안 날아가고 빠릅니다
- 04 여러분이 할 일은 상태를 바꾸는 것뿐입니다

어제

상태 바꾸고 render 를 직접 부름



오늘

상태만 바꿈



React 가

바뀐 곳을 찾아 알아서 고침

같은 기능, 다른 코드

왼쪽이 어제, 오른쪽이 오늘

어제, 직접 그리기

```
let todos = [];  
  
function addTodo(text) {  
  todos.push({ text, done: false });  
  save();  
  render();  
}
```

오늘, React

```
const [todos, setTodos] = useState([]);  
  
function addTodo(text) {  
  setTodos([...todos,  
    { text, done: false }]);  
}
```

핵심 차이 오른쪽에는 render 가 없습니다. setTodos 를 부르면 React 가 알아서 다시 그리니다

설치는 안 합니다

파일 세 개를 불러오면 끝입니다

```
<script src="lib/react.js"></script>
<script src="lib/react-dom.js"></script>
<script src="lib/babel.js"></script>

<script type="text/babel">
  // 여기에 코드를 씁니다
</script>
```

react

React 본체

react-dom

화면에 그려주는 부분

babel

JSX 를 브라우저가 알아듣게 번역

코드는 이 안에 직접 씁니다

어제와 정반대입니다

```
<!-- 안 됩니다. 파일을 그냥 열면 막힙니다 -->
<script type="text/babel" src="app.jsx"></script>

<!-- 이렇게 씁니다 -->
<script type="text/babel">
  function App() { return <h1>안녕</h1>; }
</script>
```

어제 확장은

인라인이 막혀서 파일로 빼야 했습니다

오늘은 반대

파일로 빼면 브라우저가 막습니다

이유

그냥 연 파일은 다른 파일을 못 읽게 되어 있습니다

파일을 열고 내 이름 띄우기

```
function App() {  
  return <h1>안녕하세요</h1>;  
}  
  
ReactDOM.createRoot(  
  document.querySelector("#root")  
) .render(<App />);
```

할 일

practice.html 을 열고 h1 안의 글자를 내 이름으로 바꾸기

생각해볼 것

새로고침만 하면 되나요. 어제와 다른 점은

남은 시간

08:00

02 JSX 와 컴포넌트

JSX, HTML 처럼 생긴 자바스크립트

return 안에 태그를 그대로 씁니다

01 자바스크립트 안에 HTML 을 그대로 쓸 수 있습니다

02 브라우저는 이걸 모릅니다. babel 이 번역해 줍니다

03 번역되면 평범한 자바스크립트 함수 호출이 됩니다

04 그래서 문법 규칙이 HTML 과 조금 다릅니다

```
return ( <h1>안녕</h1> );
```

괄호로 감싸는 이유는 여러 줄로 쓰기 위해서입니다

컴포넌트는 그냥 함수입니다

화면 한 덩어리를 돌려주는 함수

```
1 function Hello() {  
2   return (  
3     <div>  
4       <h1>안녕하세요</h1>  
5       <p>오늘은 React 를 씁니다</p>  
6     </div>  
7   );  
8 }
```

이름은 대문자로

소문자로 시작하면 HTML 태그로 봅니다

return 하나에

덩어리 하나. 여러 개면 div 로 감쌉니다

쓸 때는

<Hello /> 라고 태그처럼 씁니다

JSX 는 결국 함수 호출입니다

babel 이 이렇게 바꿔줍니다

```
// 우리가 쓰는 것
return <h1 className="title">안녕</h1>;

// babel 이 바꾼 것
return React.createElement("h1",
  { className: "title" },
  "안녕");
```

JSX 는 문법 설탕

읽기 편하라고 만든 걸모습입니다

babel 이 없으면

브라우저는 JSX 를 모릅니다

className 인 이유

결국 객체의 속성 이름이라서 그렇습니다

값을 넣을 때는 중괄호

JSX 안에서 자바스크립트를 쓰는 문

- 01 중괄호 안에는 자바스크립트를 씁니다
- 02 변수도 되고, 계산도 되고, 함수 호출도 됩니다
- 03 다만 값이 나와야 합니다. if 문은 못 씁니다
- 04 조건이 필요하면 물음표를 씁니다

```
const name = "홍길동";

<h1>{name}님 안녕하세요</h1>
<p>내년이면 {2026 - 2000}살</p>
<p>{name ? "로그인함" : "안 함"}</p>
```

중괄호로 값과 계산 넣기

```
function App() {  
  const name = "여기에 내 이름";  
  const year = 2000;  
  
  // TODO 아래 h1 에 이름과 나이를 넣어보세요  
  return <h1>안녕하세요</h1>;  
}
```

할 일

practice.html 에서. 이름과 2026 에서 뺀 나이를 같이 표시

생각해볼 것

중괄호 없이 name 이라고만 쓰면 어떻게 나올까

남은 시간

08:00

HTML 과 다른 점 세 개

여기서 제일 많이 막힙니다

에러 메시지	원인	해결
class 를 못 씀	class 는 자바스크립트 예약어	className 이라고 쓸 것
태그를 꼭 닫아야 함	 도 예외 없음	 처럼 슬래시를 붙일 것
덩어리는 하나만	return 에 태그 두 개를 나란히 못 둠	<div> 로 감싸거나 <> </> 로 감쌀 것

있을 때만 보여주기

if 를 못 쓰니 다른 방법을 씁니다

01 JSX 중괄호 안에는 if 문을 못 씁니다. 값이 나와야 합니다

02 보이거나 안 보이거나면 앰퍼샌드 두 개를 씁니다

03 이것 아니면 저것이면 물음표를 씁니다

04 오후 미션 코드에 둘 다 나옵니다

```
// 참일 때만 보여주기
{showClock && <Clock />}

// 둘 중 하나 고르기
{todos.length === 0
  ? <li>할 일이 없습니다</li>
  : todos.map(...)}
```


배열을 화면에 그리기

어제 쓰던 map 그대로입니다

- 01 배열에 map 을 걸어 태그 배열을 만듭니다
- 02 어제는 join 으로 글자를 만들었는데 오늘은 그럴 필요가 없습니다
- 03 중괄호 안에 배열을 넣으면 React 가 알아서 펼칩니다
- 04 다만 각 항목에 key 를 붙여야 합니다

key 는 React 가 항목을 구분하는 이름표

없으면 어느 게 바뀌었는지 몰라서 경고가 뜹니다

목록은 이렇게 그립니다

어제와 거의 같습니다

```
1  const todos = ["장보기", "운동", "공부"];
2
3  return (
4    <ul>
5      {todos.map((todo, index) => (
6        <li key={index}>{todo}</li>
7      )}}
8    </ul>
9  );
```

join 이 없음

글자를 만드는 게 아니라 태그를 만듭니다

key 를 붙임

없으면 콘솔에 경고가 뜹니다

index 를 씬

지금은 이것로 충분합니다

배열을 목록으로 그리기

```
const menus = ["김치찌개", "돈까스", "비빔밥"];

// TODO 1 menus 를 ul 과 li 로 그리세요

// TODO 2 key 를 일부러 빼고 콘솔을 보세요

// TODO 3 다시 key 를 넣으세요
```

할 일

practice.html 에서 순서대로

생각해볼 것

TODO 2 에서 콘솔에 뭐라고 떴나요

남은 시간

10:00

화면도 함수처럼 나눅니다

어제 함수 쪼갠 것과 같습니다

01 한 컴포넌트가 길어지면 읽기 힘듭니다

02 화면 덩어리마다 함수를 따로 만듭니다

03 필요한 값은 위에서 내려보냅니다

04 내려보내는 값을 props 라고 부릅니다

App

상태를 가지고 있음



props 로 내려보냄

todo 하나와 눌렀을 때 할 일



TodoItem

받은 것만 그림

props 는 함수 매개변수입니다

위에서 내려주고 아래에서 받습니다

```
1  function TodoItem({ todo, onToggle }) {
2    return (
3      <li>
4        <input type="checkbox" onChange={onToggle} />
5        <span>{todo.text}</span>
6      </li>
7    );
8  }
9
10 // 쓸 때는 태그의 속성처럼
11 <TodoItem todo={item} onToggle={handleToggle} />
```

중괄호로 받기

2일차 구조 분해입니다

함수도 내려보냄

눌렀을 때 할 일을 통째로 넘깁니다

아래에서는

받은 것만 씁니다. 상태를 안 가집니다

목록 한 줄을 컴포넌트로 빼기

```
// 지금은 App 안에 다 있습니다
{menus.map((menu, index) => (
  <li key={index}>{menu}</li>
))}

// TODO MenuItem 컴포넌트를 만들어서 빼세요
//      menu 를 props 로 받으면 됩니다
```

할 일

practice.html 에서. 화면이 그대로면 성공

생각해볼 것

컴포넌트 이름을 소문자로 하면 어떻게 될까

남은 시간

10:00

03 useState

어제 상태를 어떻게 두셨죠

let 변수 하나였습니다

01 let todos = [] 로 만들고

02 바꿀 때는 todos.push 를 하고

03 바꾼 다음에 render 를 불렀습니다

04 이 세 단계가 오늘 한 줄과 한 번의 호출로 줄어듭니다

```
const [todos, setTodos] = useState([]);
```

왼쪽이 읽는 자리, 오른쪽이 바꾸는 자리입니다

한 줄에 두 개가 나옵니다

읽는 것과 바꾸는 것

```
1  const [count, setCount] = useState(0);
2
3  // 읽을 때
4  <p>{count}</p>
5
6  // 바꿀 때
7  <button onClick={() => setCount(count + 1)}>더하기</button>
```

useState(0)

괄호 안에 처음 값입니다

count

지금 값. 읽기만 합니다

setCount

바꾸는 함수. 부르면 다시 그려집니다

버튼을 누르면 숫자가 올라가게

```
function Counter() {  
  // TODO 1  useState 로 count 를 만드세요. 처음은 0  
  
  return (  
    <div>  
      <p>{/* TODO 2  여기에 count */}</p>  
      <button>{/* TODO 3  누르면 1 올리기 */}더하기</button>  
    </div>  
  );  
}
```

할 일

practice.html 에서 순서대로

생각해볼 것

render 를 어디서도 안 불렀는데 화면이 왜 바뀔까

남은 시간

10:00

push 를 쓰면 안 됩니다

오늘 제일 많이 막히는 곳

- 01 어제는 todos.push 로 배열을 직접 고쳤습니다
- 02 React 는 배열이 바뀌었는지 주소로 비교합니다
- 03 push 는 안에 넣을 뿐 주소가 그대로입니다
- 04 그래서 React 가 바뀐 줄 모르고 화면을 안 고칩니다

새 배열을 만들어서 넘긴다

어제 버그 3번에서 본 그 주소 얘기입니다

새 배열을 만들어 넘깁니다

점 세 개를 씁니다

```
1 // 안 됩니다. 화면이 안 바뀝니다
2 todos.push({ text: "장보기", done: false });
3 setTodos(todos);
4
5 // 이렇게 씁니다
6 setTodos([...todos, { text: "장보기", done: false }]);
```

점 세 개

기존 것을 펼쳐 담고 뒤에 하나 더

결과는 새 배열

주소가 달라져서 React 가 알아챱니다

어제 버그 3

같은 얘기입니다. 복사가 아니면 같은 것

할 일 추가하기, push 와 비교하며

```
// TODO 1  먼저 push 로 해보세요. 화면이 바뀌나요  
// todos.push(text);  
// setTodos(todos);  
  
// TODO 2  이제 점 세 개로 바꿔보세요  
// setTodos([...todos, text]);
```

할 일

practice.html 에서 1번을 먼저 해보고 2번으로

생각해볼 것

1번에서 데이터는 들어갔을까요

남은 시간

10:00

가운데 것 하나만 바꾸려면

역시 새 배열을 만듭니다

- 01 체크를 뒤집으려면 그 항목 하나만 바뀌야 합니다
- 02 map 으로 전체를 돌면서 그 자리만 새 객체로 바꿉니다
- 03 나머지는 원래 것을 그대로 씁니다
- 04 결과는 새 배열이라 React 가 알아챱니다

```
setTodos(todos.map((todo, i) => (  
  i ≡ index  
  ? { ...todo, done: !todo.done }  
  : todo  
)));
```

체크는 이렇게 뒤집습니다

오후 미션에 그대로 나옵니다

```
1 function toggleTodo(index) {  
2   setTodos(todos.map((todo, i) => (  
3     i === index  
4     ? { ...todo, done: !todo.done }  
5     : todo  
6   )));  
7 }
```

`i === index` 면

새 객체를 만들어 done 만 뒤집습니다

아니면

원래 todo 를 그대로 씁니다

결과

새 배열이 나옵니다

체크하면 줄이 그어지게

```
// TODO 1 toggleTodo 를 map 으로 채우세요

// TODO 2 li 에 done 클래스를 붙이세요
//           sc-camel-class-name={todo.done ? "done" : ""}

// TODO 3 체크박스에 checked 와 onChange 를 연결하세요
```

할 일

practice.html 에서 순서대로

생각해볼 것

checked 만 넣고 onChange 를 빼면 어떻게 될까

남은 시간

10:00

04 컴포넌트의 일생

컴포넌트에도 일생이 있습니다

라이프사이클

- 01 화면에 나타납니다. 이걸 마운트라고 합니다
- 02 상태가 바뀌면 다시 그려집니다. 업데이트입니다
- 03 화면에서 사라집니다. 언마운트입니다
- 04 각 순간에 무언가 하고 싶을 때 useEffect 를 씁니다

마운트

화면에 나타남. 데이터를 불러오기 좋은 때



업데이트

상태가 바뀔 때마다. 저장하기 좋은 때



언마운트

사라짐. 뒷정리를 해야 하는 때

화면 그리기 말고 다른 일

useEffect 가 말합니다

- 01 컴포넌트 함수는 화면을 만들어 돌려주는 일만 합니다
- 02 저장하거나 불러오거나 타이머를 거는 일은 여기서 하면 안 됩니다
- 03 그런 일은 useEffect 안에 넣습니다
- 04 React 가 화면을 다 그린 뒤에 불러줍니다

useEffect(() => { ... }, [])

함수 하나와 대괄호 하나. 대괄호가 언제 부를지를 정합니다

열릴 때 한 번만

대괄호를 비워둡니다

```
1 useEffect(() => {  
2   const saved = JSON.parse(  
3     localStorage.getItem("todos")) || [];  
4   setTodos(saved);  
5 }, []);
```

빈 대괄호

처음 한 번만 부르라는 뜻

어제 **load**

그 함수가 여기로 들어왔습니다

맨 아래 호출

어제처럼 직접 부를 필요가 없습니다

대괄호 안이 전부입니다

언제 다시 부를지를 정합니다

- 01 빈 대괄호면 처음 한 번만 부릅니다
- 02 안에 값을 넣으면 그 값이 바뀔 때마다 부릅니다
- 03 대괄호를 아예 안 쓰면 다시 그릴 때마다 부릅니다
- 04 세 번째는 무한 반복이 나기 쉬워서 오늘은 안 씁니다

대괄호	언제 부르나
[]	처음 한 번만
[todos]	todos 가 바뀔 때마다
없음	그릴 때마다. 오늘은 안 씀

todos 가 바뀔 때마다 저장

어제 save 다섯 군데가 여기 하나로

```
1 useEffect(() => {  
2   localStorage.setItem("todos",  
3     JSON.stringify(todos));  
4 }, [todos]);
```

대괄호에 **todos**

todos 가 바뀔 때마다 실행

어제는

추가, 체크, 삭제마다 save 를 불렀습니다

오늘은

여기 한 군데. 빠뜨릴 수가 없습니다

새로고침해도 남게 만들기

```
// TODO 1 열릴 때 한 번 불러오기
useEffect(() => {

}, []);

// TODO 2 todos 가 바뀔 때마다 저장하기
useEffect(() => {

}, [todos]);
```

할 일

practice.html 에서. 대괄호 안이 다른 것에 주의

생각해볼 것

TODO 2 의 대괄호를 비우면 어떻게 될까

남은 시간

10:00

사라질 때 치우고 가야 합니다

정리 함수

- 01 타이머를 걸었으면 꺼야 합니다
- 02 컴포넌트가 사라져도 타이머는 안 사라집니다
- 03 useEffect 안에서 함수를 하나 돌려주면 됩니다
- 04 React 가 사라질 때 그 함수를 대신 불러줍니다

```
return () => { clearInterval(id); }
```

돌려준 함수가 뒷정리를 맡습니다

걸고, 치우고

마운트와 언마운트가 한 자리에

```
1  useEffect(() => {  
2    const timerId = setInterval(() => {  
3      setNow(new Date());  
4    }, 1000);  
5  
6    return () => clearInterval(timerId);  
7  }, []);
```

위쪽

나타날 때 타이머를 겁니다

return 아래

사라질 때 React 가 불러줍니다

timerId

무엇을 끝지 기억해두는 값입니다

정리를 안 하면 어떻게 되나

실제로 재봤습니다

- 01 시계를 꺾다 켤 때마다 타이머가 하나씩 쌓입니다
- 02 그런데 화면은 멀쩡해 보입니다. 시각이 똑같이 나옵니다
- 03 콘솔에 찍어보면 초당 실행 횟수가 늘어납니다
- 04 눈에 안 보이는 게 더 위험합니다. 메모리와 배터리를 먹습니다

꺾다 켤 횟수	초당 실행
0회	1.0번
1회	2.0번
4회	5.0번

시계를 만들고 정리까지 붙이기

```
useEffect(() => {  
  console.log("시계 시작");  
  
  // TODO 1  1초마다 setNow 를 부르는 타이머  
  
  // TODO 2  정리 함수를 돌려주기  
  //          안에 console.log("시계 정리") 도 넣을 것  
}, []);
```

할 일

practice.html 에서. 시계 끄기 버튼을 눌러 콘솔을 볼 것

생각해볼 것

TODO 2 를 빼고 껏다 켜기를 반복하면 콘솔이 어떻게 되나

남은 시간

12:00

라이프사이클 한눈에

오늘 배운 것 전부

- 01 마운트. 빈 대괄호. 불러오기
- 02 업데이트. 대괄호에 값. 저장하기
- 03 언마운트. return 으로 정리 함수. 치우기
- 04 세 개가 전부입니다

useEffect(fn, [])

마운트. 처음 한 번

useEffect(fn, [todos])

업데이트. todos 가 바뀔 때

return () => { ... }

언마운트. 사라질 때

오늘 만날 에러

오후에 반드시 한 번은 봅니다

에러 메시지	원인	해결
Objects are not valid as a React child	객체를 그대로 화면에 넣음	todo 가 아니라 todo.text 를 넣을 것
Each child should have a unique key	map 에 key 를 안 붙임	key={index} 를 붙일 것
Too many re-renders	useEffect 대괄호를 빠뜨림	대괄호를 반드시 쓸 것

어제 만든 것을 React 로 다시

화면은 똑같습니다. 안이 달라집니다

기본, 전원 완료 목표

- 할 일 추가하기, setTodos 로
- 체크 뒤집기, map 으로
- 저장하고 불러오기, useEffect 로
- 시계에 정리 함수 붙이기

심화, 기본을 마친 뒤

- 완료한 것 숨기기
- 컴포넌트 더 나누기

오늘 안 깐 것들은 뭐가

npm 과 빌드 도구

01 지금 babel 이 브라우저에서 매번 JSX 를 번역하고 있습니다

02 그래서 처음 열 때 조금 느립니다

03 실무에서는 미리 번역해두고 결과만 올립니다

04 그걸 해주는 게 빌드 도구고, 받아오는 도구가 npm 입니다

오늘

브라우저가 열 때마다 번역



실무

미리 번역해서 결과만 올림



도구

npm 으로 받고 빌드해서 배포

**지금 만든 페이지는 내 컴퓨터에만 있습니다. 이 주소를 친구
에게 보내려면 무엇이 더 있어야 할까요**

오늘 배운 것

세 줄로 정리

01 JSX 와 컴포넌트로 화면을 나누기

02 useState 로 상태를 두고 render 없이 다시 그리기

03 useEffect 로 불러오고, 저장하고, 뒷정리하기

TOMORROW

오전, 배포 이론과 GitHub Pages

오후, 올리고 다듬고 마무리